How can coding agents support scalable development?

Lecture 2 · Establish

More implementation capacity makes the feedback system more important.

The development system scales too

A web project is changed by a connected system of:

- people and agents;
- instructions and context;
- tools and environments;
- tests and review;
- ownership and correction paths.

Faster code production does not guarantee faster accepted change.

Constrain → generate → validate

Constrain

State the behavior, boundaries, commands, and prohibited effects.

Generate

Let the agent propose and implement a bounded change.

Validate

Check the artifact independently and feed concrete failures back into the loop.

The loop ends with accepted evidence—not generated output.

A project harness makes work inspectable

The repository should answer five questions without oral rescue:

1. What behavior is required?
2. Where do important decisions belong?
3. How is the application run?
4. Which checks establish acceptance?
5. Who owns correction when a result is wrong?

README, feature specification, architecture note, agent instructions, and executable commands form one control surface.

Validation should climb a ladder

1. Static checks catch local representation errors.
2. Unit and component checks protect bounded behavior.
3. Integration checks exercise contracts and persistence.
4. Browser journeys test user-visible outcomes.
5. Operational evidence tests the running system under declared conditions.

No single rung proves the whole application.

Authority should shrink as consequences grow

An agent may:

- **propose** when intent is ambiguous;
- **prepare** when a human must approve a consequential action;
- **execute** when scope, evidence, reversibility, and limits are clear.

Provenance records who or what produced an artifact. It does not prove correctness or transfer accountability.

Improve the feedback loop, not just the prompt

When an agent struggles, ask which system property is weak:

- the requirement is ambiguous;
- the relevant context is hard to find;
- the change boundary is unclear;
- checks are slow or incomplete;
- failure output does not explain what to correct.

Prompt changes help only when the prompt is the actual bottleneck.

Establish checkpoint

Make implementation A reviewable through:

- reproducible setup and run commands;
- agent instructions and declared constraints;
- a maintained feature and architecture record;
- deterministic validation commands;
- one decision record and a compact AI-use log.

Question to carry forward: What can the project now measure repeatedly?